Digital Vector · VCE Applied Computing | Unit 1 · Area of Study 2 · Programming · KK04

Requirements, Constraints & Scope

Before a single line of code is written, four questions decide whether your software succeeds: what must it do, how well must it do it, what limits hem you in, and where is the edge of the job. Read them wrong and you build the wrong app perfectly.

What this key knowledge covers
This summary distils everything you need for **KK04 — Requirements, Constraints & Scope** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Why this dotpoint matters

In Outcome 2 you **interpret a teacher-provided brief** and build a working program in an object-oriented language. You are not inventing the problem — you are decoding it. Every brief, no matter how short, can be split into the same four buckets. VCAA literally asks you to **read a short case study and write a table** listing the functional and non-functional requirements , the constraints, and the scope. Get fluent at that table and you've nailed a guaranteed chunk of marks.

What the solution must **do** — its actions and behaviours.

How well it must do it — the qualities and standards.

A **limit** you must work within — not chosen, imposed.

The **boundary** — what's in the job and what's out.

Ordering a pizza

You don't need to be a developer to already understand all four ideas — you use them every time you order food. Picture phoning a pizza shop.

"Make a large pepperoni, cut into 8 slices."

"…and I want it *hot* and here in *30 minutes*."

"The shop has one oven and shuts at 9pm."

"Pizza only — no, you can't add a haircut."

Notice the relationships: the constraint (one oven, closes at 9) is exactly why the non-functional "30 minutes" is hard to meet, and it's why "also wash my car" falls out of scope. The four lenses aren't separate facts — they pull on each other.

FUNCTIONAL What it must do

A *functional requirement* describes a specific **action, behaviour or function** the software must perform. The cleanest test: **could you tick a yes/no box that says "the software did this"?** If yes, it's functional.

Functional requirements are almost always written as a **verb the program performs** on some **data**: *calculate*, *store*, *display*, *sort*, *validate*, *add*, *delete*.

PixelBoard — functional requirements

- ▸ Let a player **enter three initials** after a game ends.
- ▸ **Store** the player's score with their initials and the date.
- ▸ **Sort** all saved scores from highest to lowest.
- ▸ **Display the top ten** scores on the leaderboard screen.
- ▸ **Reset** the leaderboard when the teacher enters a passcode.

Each one is a behaviour you could demonstrate to the Tech Club and they could nod "yep, it did that." Notice how testable they are — that testability is what makes them functional.

NON-FUNCTIONAL How well it must do it

A *non-functional requirement* doesn't add a new behaviour — it puts a **quality standard** on the behaviours you already have. Same job, higher bar. These describe the **experience and the standard**, not the function.

Most non-functional requirements are an **"-ility"**. Memorise the family and you'll spot them instantly:

Easy to learn and use. PixelBoard: a Year 7 can enter their initials with no instructions.

Works consistently without crashing. No saved score is ever lost between games.

Fast enough. The leaderboard redraws in under 1 second.

Protected from misuse. Only the teacher passcode can wipe the board.

Runs in different places. Runs on the cabinet's old browser and a laptop.

Easy to fix; survives bad input. Typing "12" for initials doesn't crash it.

Sort the lens: drag each spec into its bucket

Here are real lines from a PixelBoard brief. Drag each into the right bucket — or tap a chip then tap a bucket on touch screens. You're training the reflex examiners reward.

CONSTRAINT The limits you can't change

A *constraint* is a **restriction or limitation** imposed on the solution — by money, technology, the law, time, or people. The key word is **imposed**: you don't choose a constraint, you're stuck with it, and it shapes every decision. VCAA groups them into recognisable types:

SCOPE Where the job stops

Scope defines the **boundary** of the solution — the line between what the software **will** do (in scope) and what it **won't** (out of scope). Writing scope is mostly about saying **no** on purpose, so everyone agrees on the edges before you build.

In scope ✓

- ✓ Saving a score with three initials
- ✓ Showing the local top-ten
- ✓ A teacher reset with a passcode

Out of scope ✗

- ✗ Online accounts & global leaderboards
- ✗ Syncing scores between machines
- ✗ Running the arcade game itself

Examine PixelBoard through four lenses

One product, four ways of looking at it. Switch lenses to recolour the cabinet and reveal what that lens captures. The point: **the same software answers all four questions at once** — they're views of one thing, not four different things.

The behaviours the cabinet performs — each one testable yes/no.

- Enter three initials
- Store score + date
- Sort high → low
- Display the top ten
- Teacher passcode reset

The spec table examiners want

When VCAA hands you a case study, the safest move is to **build the four-column table**. Here's the full PixelBoard brief reduced to one. If you can produce this from a paragraph of text, you can answer almost any question on this dotpoint.

Name that constraint type

Constraints trip people up because you also have to name the *type*. Read each line from a new brief (a school excursion-booking app) and pick the constraint type. Instant feedback explains the call.

Six mistakes that quietly lose marks

Writing "the app is fast" as a functional requirement. Speed is a standard, not an action.

"It should be user-friendly and fast." Examiners can't award a tick for a wish.

Listing "must use free software" as a non-functional requirement.

Just re-listing the functional requirements when asked for scope.

Adding "and it logs players in with Google" because it sounds impressive.

Answering a 3-mark "explain" with a label and nothing else.

Command words & the P·E·L move

Half of “losing marks” isn’t not knowing — it’s under-writing. Match your answer length to the **command word**:

Just name it. One line. No explanation needed.

Name it *and* add detail — what it is, what it covers.

Say *why / how* — give a reason and connect it to the case.

Make a case with evidence, or weigh two options against each other.

For “describe / explain” marks, the reliable structure is **P·E·L**:

P **Point** — name the concept clearly. E **Evidence** — tie it to the specific case study. L **Link** — connect back to the question or its impact.

Exam questions — see the weak vs strong answer

Each question shows a weak response with examiner annotations, a strong model answer, and the marking guide. Try it yourself first, then reveal.

- 1 mark: definition identifies a function/action/behaviour the software *does*.
- 1 mark: a correct, behaviour-based example tied to PixelBoard (e.g. sort scores, store a score, display top ten).
- 1 mark: correct definition of non-functional requirement (a quality/standard of the product).
- 1 mark: correct definition of constraint (an imposed limitation on development).
- 1 mark: a valid, distinct PixelBoard example of each.
- 1 mark: an explicit point of difference (product target vs build limitation).
- 1 mark: identifies that scope defines an in/out boundary.
- 1 mark: explains it prevents scope creep / uncontrolled feature growth.
- 1 mark: links the benefit to PixelBoard’s context (shared understanding / meeting the week-8 deadline).
- 1 mark each for two correct functional requirements (behaviours: save score, show top ten).
- 1 mark each for two correct non-functional requirements (qualities: reliability, usability).
- No mark if a quality is listed as functional or vice-versa.
- 1 mark: functional requirements correct and complete (incl. daily total).
- 1 mark: at least one genuine non-functional requirement.
- 1 mark: identifies the technical (printer) constraint.
- 1 mark: identifies time and/or legal constraint.
- 1 mark: scope states both in *and* out (refunds out of scope).

The whole dotpoint in four lines

- FUNC **Functional** = what it must **do**. A behaviour you can tick yes/no. Written as a verb on data (sort, store, display).
- NON-F **Non-functional** = how **well**. A quality bar — usually an “-ility” (usability, reliability, security). Make it measurable.
- CONST **Constraint** = an imposed **limit** (economic, technical, legal, social, time). A fence around how you build.
- SCOPE **Scope** = the **boundary**. State what’s in *and* what’s out. Guards against scope creep.

Key terms

Term	Meaning
Lens	PixelBoard specification
Functional	Enter initials · store score+date · sort high→low · display top ten · teacher passcode reset
Non-functional	Usable by Year 7s unaided · redraw under 1 s · never loses a score · survives bad input
Constraints	$0 budget (free tools) · runs on the cabinet's old browser · no minors' full names · done by week 8
Scope	**In:** local save, top-ten, passcode reset. **Out:** online accounts, cross-machine sync, the game itself

Common traps & misconceptions

Exam trap
⚠ **The #1 exam slip.** "Stores the score" is functional (a behaviour). "Stores the score *reliably so none are ever lost*" is non-functional (a quality on that behaviour). The same sentence can contain both — the verb is functional, the standard is non-functional.

Exam trap
⚠ **Constraint vs non-functional — the tricky pair.** They feel similar because both are "not a behaviour." The split: a non-functional requirement is a *quality you aim to achieve* in the finished product ("loads in under 1 second"). A constraint is a *limit on how you're allowed to build it* ("must use only free software," "must be done by week 8"). Ask: **is this a target for the product, or a fence around the project?** Target → non-functional. Fence → constraint.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Distinguish · 2 marks

Distinguish between a **constraint** and a **requirement** for a software solution.

✗ A weak answer

A requirement is what you need and a constraint is a problem. *(→ "Problem" is wrong; constraints are limits, not faults — 0–1.)*

✓ A strong answer

A **requirement** is something the finished solution **must do or be** (e.g. "calculate the weekly total"). A **constraint** is a **limitation the team must work within** while building it (e.g. "must run on the school's existing tablets" or "must be finished in four weeks"). Requirements describe the target; constraints describe the boundaries.

How the marks are awarded

- 1 mark — requirement = what the solution must do/be.
- 1 mark — constraint = a limiting factor the developer works within.

Q2. Explain · 3 marks

Explain the difference between **functional** and **non-functional** requirements, using one example of each for a quiz app.

✗ A weak answer

Functional is what it does and non-functional is the other stuff like colours. *(→ "Other stuff" is too loose; "colours" alone is unconvincing — 1.)*

✓ A strong answer

A **functional requirement** describes an action the solution must perform — e.g. "the quiz app must mark each answer and show a final score." A **non-functional requirement** describes a **quality** of how it performs — e.g. "each question must load in under one second" or "the interface must be usable on a phone screen." Functional = what it does; non-functional = how well it does it.

How the marks are awarded

- 1 mark — functional defined with a valid example.
- 1 mark — non-functional defined as a quality/standard.
- 1 mark — a valid, measurable non-functional example.

Revision checklist

- FUNC Functional = what it must do. A behaviour you can tick yes/no. Written as a verb on data (sort, store, display).
- NON-F Non-functional = how well. A quality bar — usually an “-ility” (usability, reliability, security). Make it measurable.

- CONST Constraint = an imposed limit (economic, technical, legal, social, time). A fence around how you build.
- SCOPE Scope = the boundary. State what’s in and what’s out. Guards against scope creep.